MG MUSALLAM GROUP · DELIVERY OPERATIONS

MG-GO Admin Console

Technical Stack Overview

The exact languages, frameworks, services and versions the platform runs on — console, functions, database, edge, hosting and the rider app — with the file every figure was read from.

Next.js 16 · React 19 | Flutter 3.44 · Dart 3.12 | Firebase · Firestore

MG-GO ADMIN CONSOLE · TECHNICAL STACK OVERVIEW

Contents

1. System architecture

- 1.1 The four layers
- 1.2 A request, end to end
- 1.3 Application architecture at a glance

2. Frontend

- 2.1 Core framework and language
- 2.2 UI, styling and design system
- 2.3 Internationalisation
- 2.4 Data, forms and validation
- 2.5 Maps, geo and visualisation
- 2.6 Documents, media and AI
- 2.7 Observability
- 2.8 Console surface

3. Backend

- 3.1 Cloud Functions (the application backend)
- 3.2 The three router lanes
- 3.3 Cloudflare Workers (edge services)
- 3.4 The live-tracking path
- 3.5 Next.js server layer
- 3.6 Authentication and authorisation

4. Database

- 4.1 Engine and architecture
- 4.2 Document model
- 4.3 Domain groups (142 collections)
- 4.4 Indexes and read economics
- 4.5 Legacy database
- 4.6 File storage

5. Hosting and deployment

- 5.1 Hosting matrix
- 5.2 Regions and data residency
- 5.3 Vercel configuration
- 5.4 Cloud Functions deployment
- 5.5 Release checklist

6. Rider application (MG-GO)

- 6.1 Runtime and language
- 6.2 Architecture inside the app
- 6.3 Packages in use
- 6.4 Security and integrity controls
- 6.5 Backend integration
- 6.6 Build and release
- 6.7 Tests

7. Engineering standards

- 7.1 Types and guards
- 7.2 Tests
- 7.3 Change control

8. Source code and documentation

- 8.1 Source delivery
- 8.2 Delivered manuals
- 8.3 Architecture documents
- 8.4 API surface

9. Provenance

- 9.1 Where each figure was read
- 9.2 Document control

FIGURES AND PLATES

1 The four layers
2 One name, one lane
3 The admin console, high level
4 The rider app, high level
5 The live path
6 Regions and residency

VERSION
Version 1.5 · 11 Nov 2026

READER
Client technical stakeholders, onboarding engineers, procurement

REBUILD
npm run docs:stack

This document names the exact languages, frameworks, services and versions the platform runs on, and points at the file each figure was read from. Every number below was re-read from the source tree on **11 November 2026**; nothing is estimated, and nothing is carried forward from an earlier edition without re-checking it.

The platform is two applications and one managed backend:

- **MG-GO Admin Console** — a Next.js App Router web console for staff, deployed on Vercel, rendered in English and Arabic.
- **MG-GO Rider Application** — a Flutter Android app for riders, distributed through Google Play.
- **Firebase** — Cloud Functions (gen 2) for the application backend, Firestore for data, Auth for identity, FCM for push.

ADMIN CONSOLE
Next.js 16.2.6
React 19.2.4 · TypeScript 5 · Tailwind 4

CONSOLE SURFACE
41 areas
15 API route groups · 13 cron jobs

RIDER APP
Flutter 1.1.26+118
Dart ^3.11.0 · Android only

APPLICATION BACKEND
204 functions
3 router lanes · 106 RPC modules

DATABASE
Firestore 142 collections
201 composite indexes

LOCALISATION
12,591 keys × 2
English · العربية (full RTL)

HOSTING
Vercel bom1 · Cloud Functions me-central1
Firestore me-central2

EDGE
3 Cloudflare Workers
FleetRoom Durable Object · 4 Hz

STORAGE
2 R2 buckets
dpd-private-prod · dpd-gps-prod

ACCESS CONTROL
172 slugs in code
174 synced · 22 categories

TESTS
60+ suites
node:test · flutter test · RTL overflow guards

OBSERVABILITY
Sentry 10.55
dpd-admin · flutter-mussalam

> **Reading the numbers.** A count is a fact about a commit, not a promise. The collection count, the index count and the function inventory move whenever a module ships, and each is re-derived rather than remembered. Section 9 records exactly which file every figure came from, so the next edition can be checked rather than trusted.

1 System architecture

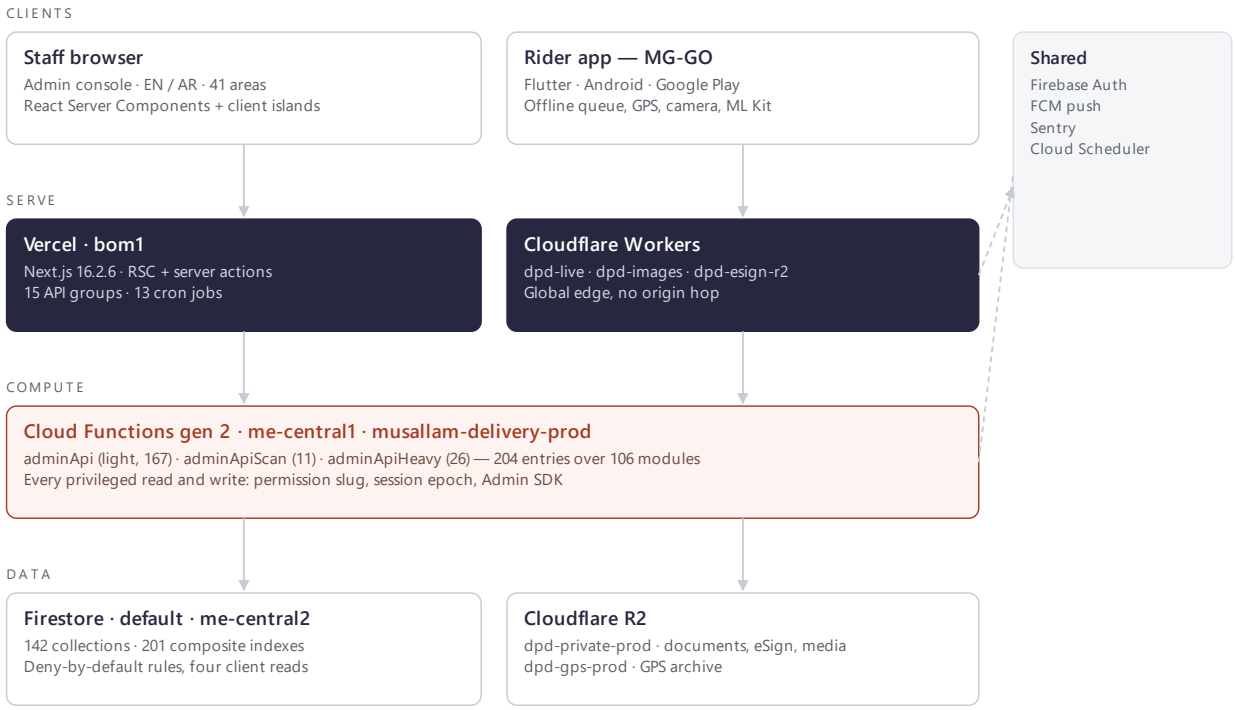


Figure 1. The four layers. The console holds no database, the functions render no HTML, and the Workers never hold the only copy of anything.

The platform is deliberately layered, and each layer has one job. The console never owns a database; the functions never render HTML; the Workers never hold the only copy of anything.

1.1 The four layers

Layer	Runs on	Owns
Client	Staff browser · rider phone	Rendering, input, local cache and queued offline writes
Serve	Vercel (`bom1`) · Cloudflare edge	HTML/RSC, API routes, server actions, static assets, WebSocket fan-out
Compute	Cloud Functions gen 2 (`me-central1`)	Every privileged read and write, permission checks, payroll maths, PDF composition
Data	Firestore (`me-central2`) · R2	The only durable copy of records and files

Three rules hold this together, and each one is a decision rather than a habit:

1. **The browser never writes.** Firestore rules are deny-by-default; every mutation goes through a server action or a callable that has already checked a permission slug. `firestore.rules` allows exactly four client reads — a rider's own `drivers` , `rider_signals` , `wrong_actions` rows, and the public `app_settings/1` row.
2. **The console holds no data.** Vercel rendering is stateless. Reading the platform from a cold `bom1` instance costs nothing in consistency because there is nothing local to be stale.
3. **One name, one lane.** The client and the server resolve a callable's lane from the same file, so a function cannot be deployed somewhere the browser does not look for it.

1.2 A request, end to end

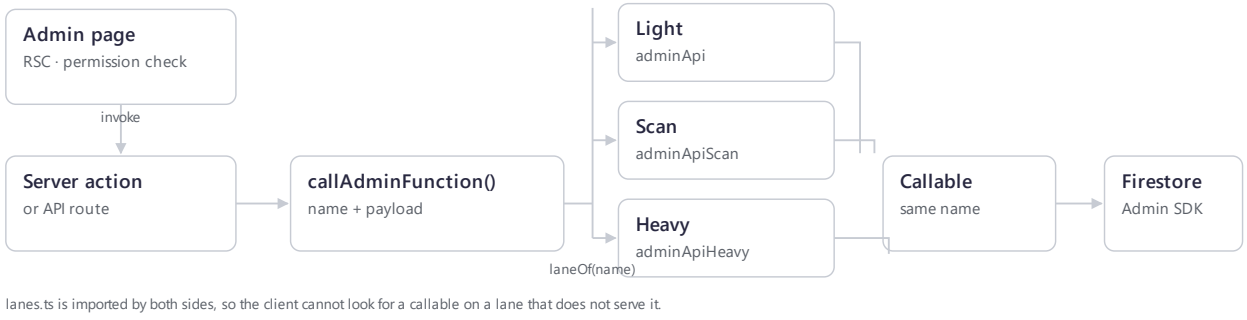


Figure 2. One name, one lane. The client resolves the lane from the same file the server does, and each lane is a separate Cloud Run service with its own limits.

A list page or a save follows the same six steps:

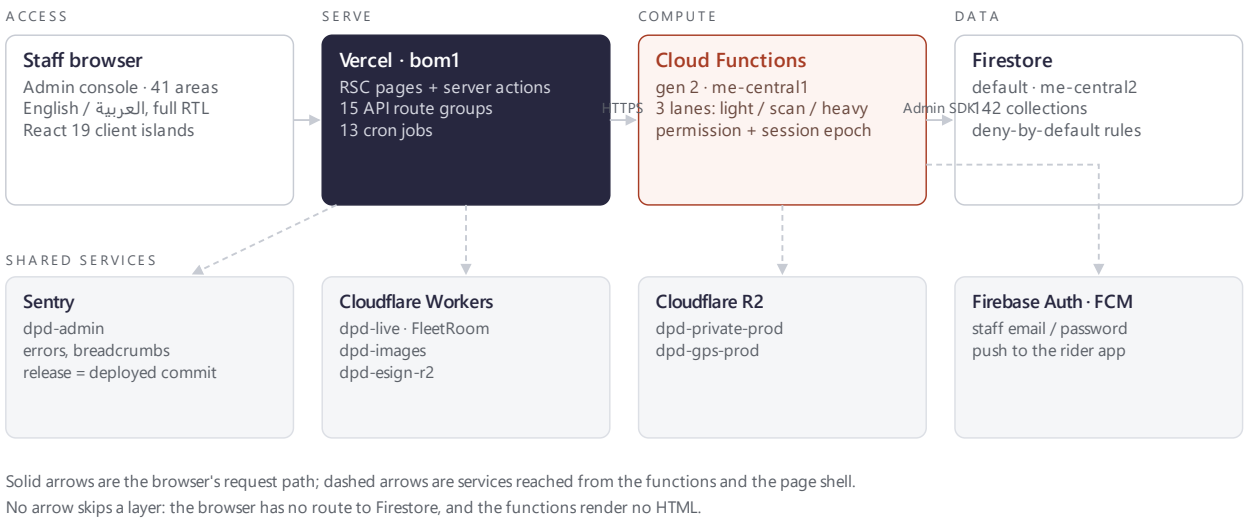
1. The page (a React Server Component) checks the permission slug, then renders.
2. A filter change or a save calls a **server action** or an API route.
3. The action calls `callAdminFunction(name, payload)`.
4. `laneOf(name)` in `functions/src/router/lanes.ts` resolves the lane.
5. The callable on that lane re-checks the session, the epoch and the slug.
6. The Admin SDK reads or writes Firestore; the result returns as JSON.

The lane split is the reason a payroll scan cannot starve a single-document save: the three lanes are separate Cloud Run services with their own concurrency, memory and instance ceilings.

1.3 Application architecture at a glance

Two diagrams, one per application. They are deliberately **high level**: each box is a layer or a managed service, not a module, and every arrow is a real call direction taken from the source. The point is to answer "what talks to what" in ten seconds — the module-by-module detail lives in the source tree and in `AGENTS.md`.

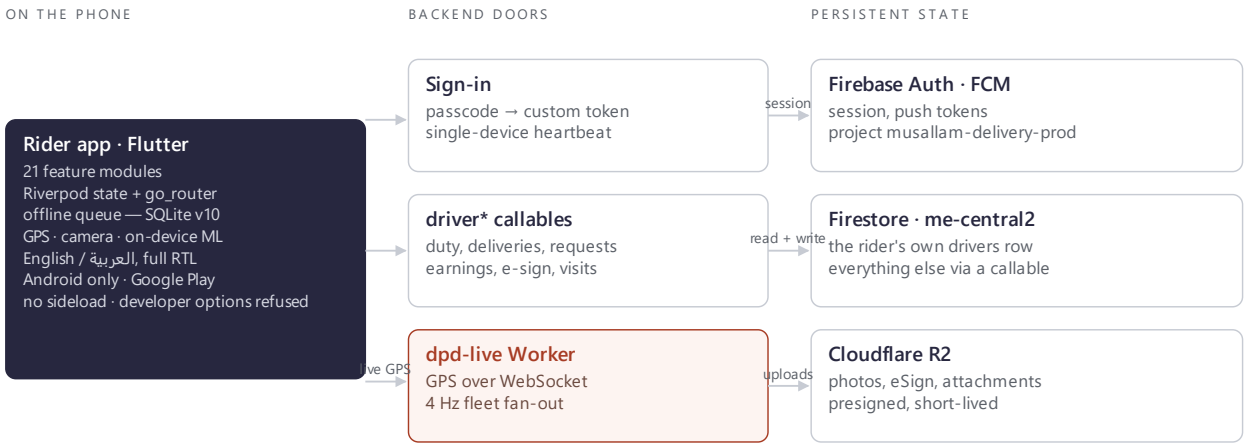
Admin console. Staff work in a browser; every privileged action passes through Vercel and then a Cloud Function, which is the only layer holding credentials to the data.



Solid arrows are the browser's request path; dashed arrows are services reached from the functions and the page shell.
No arrow skips a layer: the browser has no route to Firestore, and the functions render no HTML.

Figure 3. The admin console, high level. Four layers — access, serve, compute, data — and the shared managed services each of them uses.

Rider app. The app is offline-first: it renders and queues locally, then reaches the same backend through three different doors — a passcode-derived Firebase session, the `driver*` callables, and the edge Worker for live GPS.



Offline first: a duty change, a pickup or a completed delivery is written to the local SQLite queue and drained when connectivity returns, so a delivery finished in a dead zone is not lost. The queue is the reason the app can be used in a basement, and the reason every write carries an id so a retry cannot double count.

Documents leave the phone only through the admin API's presigned-upload route — the app never holds a bucket key. Attachment and eSign PDFs are read back the same way, by short-lived signed URL.

Figure 4. The rider app, high level. Offline-first on the phone, and three separate doors to the same backend — session, callables, edge.

Three rules hold both diagrams together, and each is a decision rather than a habit:

1. **The browser never writes.** Firestore rules are deny-by-default; every mutation goes through a server action or a callable that has already checked a permission slug. `firestore.rules` allows exactly four client reads — a rider's own `drivers`, `rider_signals` and `wrong_actions` rows, and the public `app_settings/1` row.
2. **The console holds no data.** Vercel rendering is stateless, so there is nothing local to be stale behind a cold instance.

3. **One name, one lane.** The client and the server resolve a callable's lane from the same file, so a function cannot be deployed somewhere the browser does not look for it.

2 Frontend

The console is one Next.js App Router application. It is an internal back-office product, not a marketing site: **41 console areas** live under `src/app/[locale]/(dashboard)`.

2.1 Core framework and language

Item	Value	Source
Framework	**Next.js 16.2.6** (App Router, Turbopack)	`package.json`
View library	**React 19.2.4** / React DOM 19.2.4	`package.json`
Language	**TypeScript 5.x**, strict, `tsc --noEmit` in CI	`tsconfig.json`
Compiler extras	React Compiler rules enforced through ESLint	`eslint.config.mjs`
Node engine	`>=20.9.0 <25` locally · `.nvmrc` = **22** · Vercel runtime **24.x**	`package.json`, `.nvmrc`, `.vercel/project.json`
Rendering	React Server Components by default; client components only where interaction needs them	`src/app/**`

2.2 UI, styling and design system

Item	Value
CSS	**Tailwind CSS v4** (`@tailwindcss/postcss`), `tw-animate-css`
Component base	**shadcn/ui 4.7**, `@base-ui/react` 1.4, Radix-style primitives
Variants / merge	`class-variance-authority` , `clsx` , `tailwind-merge`
Icons	`lucide-react` 1.16 — one icon on every chip, toggle and section label
Toasts	`sonner` 2.0.7
Theming	`next-themes` 0.4.6 (light / dark), one locked shell
Fonts	Gotham (Latin) + GE SS TWO (Arabic), shipped as local OTF — no runtime fetch

One house system governs every screen, and it is written down rather than implied: footer-first modals through `AppModalFooter` , `h-9` compact controls, semantic colour for state, `isFitForViewport` density tuned to a 14-inch laptop. The rules live in `.cursor/rules/ui-system.mdc` ; the tokens and rationale live in `docs/DESIGN_SYSTEM.md` .

2.3 Internationalisation

- **next-intl 4.12** with locale-prefixed routes (`/en/...` , `/ar/...`).
- **Arabic is a first-class RTL locale.** Layout is logical (`start` / `end`), not mirrored by a second stylesheet.
- Catalogues: `src/messages/en.json` and `src/messages/ar.json` , **12,591 leaf keys each**.

- `npm run test:i18n` resolves every `t("…")` literal in the source against both files and fails on a missing key **or** a key that resolves to a group. A raw key is a perfectly valid string, so no type check can catch it; this guard is the only thing that can.

2.4 Data, forms and validation

Item	Value
Server state	**TanStack Query 5.100** — cache, invalidation, optimistic patches, `keepPreviousData`
Virtualisation	`@tanstack/react-virtual` 3.14 for long payroll and roster grids
Forms	**React Hook Form 7.76** + `@hookform/resolvers`
Validation	**Zod 4.4** at the write boundary, mirrored by server-side guards
Drag and drop	`@dnd-kit/core`, `@dnd-kit/sortable`, `@dnd-kit/utilities`
Tables / export	`exceljs` 4.4 and `xlsx` 0.18 — workbooks built in the browser from the rows on screen

The query layer is what makes a 900-rider list usable: filters live in the URL, the pager is keyset-based, and a mutation invalidates through factories in `src/lib/query/query-keys.ts` rather than raw arrays, so a screen cannot be refreshed with a stale key.

2.5 Maps, geo and visualisation

Item	Value
Map rendering	**deck.gl 9.3** (`@deck.gl/extensions`, `@deck.gl/google-maps`) for the 1 Hz fleet canvas
Tile / vector maps	`leaflet` 1.9 + `react-leaflet` 5.0, MapTiler or OpenStreetMap
Zone drawing	`@geoman-io/leaflet-geoman-free` 2.19
Geo maths	`@turf/turf` 7.3, `h3-js` 4.5 (Uber H3 blocks for painted zones)
Markers	`@googlemaps/markerclusterer` 2.6 for the V1 map
Charts	`recharts` 3.10

2.6 Documents, media and AI

Item	Value
PDF composition	`pdf-lib` 1.17 — eSign sheets, signature stamping, signed-copy merge
Server-side rendering to PDF	`puppeteer-core` 24.43 + `@sparticuz/chromium` 147
QR codes	`qrcode.react` 4.2 (visit tickets, booking codes)
Assistant	`ai` 7.0 + `@ai-sdk/react` 4.0, `openai` 7.25

2.7 Observability

Sentry 10.55 (`@sentry/nextjs`) — org `vizsoft-global`, project `dpd-admin`, tunnel route `/monitoring`, with separate client, server and edge configs. Session replay is off; errors, breadcrumbs and release tags are on, and the release is the deployed commit.

2.8 Console surface

The 41 areas, by job:

Job	Areas
People	`drivers`, `drivers/groups`, `driver-devices`, `compliance`, `document-expiry`, `worktime`, `driver-shifts`
Operations	`live-tracking`, `live-tracking-v2`, `deliveries`, `dpd-verification`, `order-exceptions`, `zones`, `restaurants`, `partners`, `delivery-rules`, `incentive-rules`
Money	`payroll`, `payroll-hub`, `earnings`, `earnings-calculation`, `payouts`, `performance`
EmployeeDesk	`employeedesk` (requests, visits, e-signature, reports, settings)
Fleet	`fleet`, `vehicles`, `fuel`, `assets`
Clients	`client-portal-v2`, `operations-dashboard-v2`
Platform	`dashboard`, `assistant`, `notifications`, `support`, `requests`, `settings`

3 Backend

3.1 Cloud Functions (the application backend)

Item	Value
Platform	**Firebase Cloud Functions, 2nd generation** (Cloud Run)
Runtime	**Node.js 22** (`firebase.json` → `runtime: nodejs22` , `engines.node: "22"`)
Language	**TypeScript 5.9**, compiled to `functions/lib` by `tsc`
Region	`me-central1` — the closest region to Kuwait
Project	`musallam-delivery-prod`
SDKs	`firebase-admin` 13.10, `firebase-functions` 6.0, `@aws-sdk/client-s3` + `s3-request-presigner` (R2), `pdf-lib`

Measured inventory: 204 deployed function entries and **106 RPC modules** under `functions/src/rpcs` (test files excluded). The inventory is machine readable in `functions/router-inventory.json` , which also records each entry's memory, CPU, timeout and lane.

3.2 The three router lanes

Rather than deploying hundreds of callable URLs, the admin surface is served by three shared routers. The client resolves a name to a lane from the same file the server uses.

Lane	Export	Entries	Purpose
Light	`adminApi`	**167**	Single-document reads, saves, small mutations
Scan	`adminApiScan`	**11**	Collection scans, dashboards, payroll snapshots, reports
Heavy	`adminApiHeavy`	**26**	Bulk delete, purge, batch compose, retention sweeps

Why three and not one: Cloud Run scales per service. A payroll month scan that takes 20 seconds must not occupy an instance that a picker's dropdown needs for 40 milliseconds. Lane limits are set independently — the light lane keeps a warm instance, the scan lane gets memory and a long timeout, the heavy lane gets one concurrency.

> **Deployment consequence.** A callable deployed by name is not on the path the browser takes. Changing a routed RPC means redeploying **the lane**, and a changed payload contract means redeploying every lane that serves it. This has bitten before; the checklist in §5.4 exists because of it.

3.3 Cloudflare Workers (edge services)

Three Workers, all under Cloudflare account `04c485faa34d67d1758b61465202e9ce` :

Worker	Role	Bindings	Compatibility
`dpd-live`	Live-tracking ingest and fan-out. A **Durable Object (`FleetRoom`)** holds the Kuwait fleet in SQLite, broadcasts positions over WebSocket and flushes durable state.	`FLEET` Durable Object, `GPS_ARCHIVE` R2 (`dpd-gps-prod`)	`2025-04-01`
`dpd-images`	On-the-fly image transformation and delivery for private assets.	Cloudflare Images, R2 `dpd-private-prod`	`2024-12-01`
`dpd-esign-r2`	Signed-URL and read proxy for eSign PDFs held in R2.	R2 `dpd-private-prod`	`2024-12-01`

`dpd-live` runs one room for the whole country today; the room key is a sharding seed that is already in place, so a second room is a configuration change rather than a rewrite.

3.4 The live-tracking path

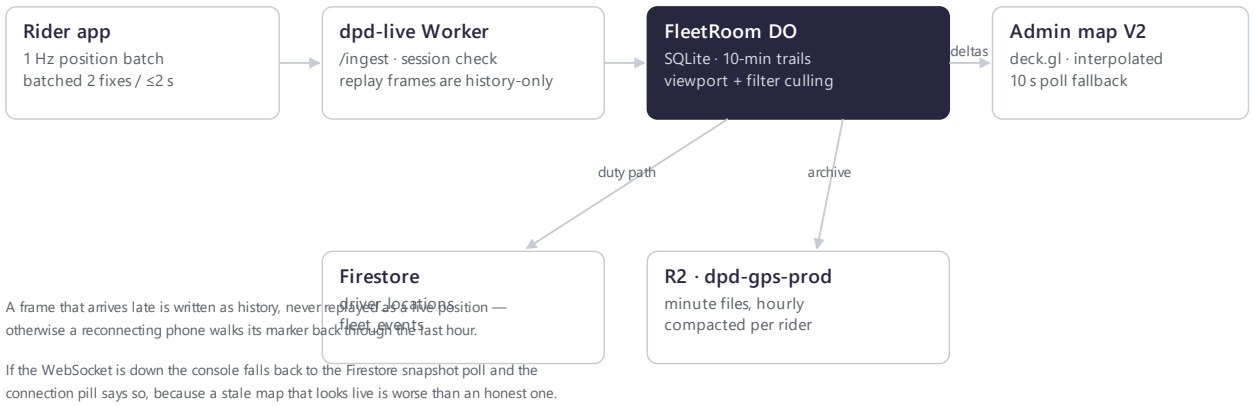


Figure 5. The live path. The Durable Object holds the fleet, the admin sees deltas, and durable state is flushed so a room restart is not a data loss.

1. The rider app posts a position batch to `/ingest` on `dpd-live` at up to 1 Hz.
2. `FleetRoom` validates the rider's session, updates its in-memory fleet, and appends a trail entry (10 minutes, trimmed).
3. Connected admin sockets receive a delta frame keyed to their viewport and filters — interest management is what keeps 500 pins affordable.
4. Positions are flushed to R2 for history and, on the duty path, to Firestore so a record survives a room restart.
5. The admin map renders with deck.gl and interpolates between fixes, so a marker glides instead of jumping.

If the edge is unreachable the console falls back to a Firestore snapshot poll and says so in the connection pill — a rail that is down must never look live.

3.5 Next.js server layer

Work that belongs next to the renderer, not in a function:

- **15 API route groups** under `src/app/api/*`: `admin`, `assistant`, `build-id`, `cron`, `deliveries`, `driver-app`, `driver-uploads`, `drivers`, `esign`, `launcher`, `live-tracking-v2`, `menu-icons`, `notifications`, `order-recon`, `storage`.
- **13 cron jobs** (`vercel.json`), all idempotent and all safe to re-run:

Job	Schedule	Job	Schedule
`locations-cleanup`	every 2 min	`expire-stale-pickups`	every 15 min
`notifications-dispatch`	every 5 min	`roster-alerts`	every 15 min
`notifications-automations`	every 5 min	`requests-sweep`	hourly
`attendance-auto-checkout`	every 5 min	`driver-ops-retention`	daily 01:20
`dpd-shift-notices`	every 5 min	`driver-telemetry-retention`	daily 01:40
`esign-batch-drain`	every 5 min	`performance-rollup`	daily 02:00
`storage-uploads-cleanup`	daily 04:00		

- **Server Actions** for form writes, each gated on a permission slug before it touches data.

3.6 Authentication and authorisation

- **Firebase Authentication** — staff email/password, rider passcode. Supabase Auth is retired and unreachable.
- Authorisation is a **permission-slug catalog**: **172 slugs in code** (`src/lib/auth/permission-catalog.ts`), **174** in the synced `admin_permissions/catalog` document, across **22 categories**.
- A session resolves its slug set once, then caches it against `auth_epochs/current` — a counter bumped by every role, tick, menu or settings write. A grant therefore reaches a signed-in admin on their next request, not their next login.
- Four ways to be staff, and the difference matters:

Account	Resolves to	Notes
Super Admin	Every slug	`admin_roles.is_super_admin`; bypasses the slug check
Manager — Full access	Every slug in the catalog	`access_kind: "manager"`
Manager — User	Only the explicit ticks	`access_kind: "user"` + `admin_user_permissions`
Partner	Two areas, read-only	`access_kind: "partner"`, locked to one project key

A manager's catalog read is unioned with the code catalog at runtime, so a slug added in code can never be invisible to Full access before the documents are synced.

4 Database

4.1 Engine and architecture

Item	Value
Engine	**Google Cloud Firestore**, Native mode
Edition	Enterprise
Database id	`default` (a named database, not `(default)`)
Project	`musallam-delivery-prod`
Location	`me-central2`
Rules	`firestore.rules` — deny-by-default, four client reads, staff writes via the Admin SDK
Indexes	`firestore.indexes.json` — **201 composite indexes** + **5 field overrides**
Collections	**142** declared in `src/lib/firebase/db.ts` (`COLLECTIONS`)

4.2 Document model

Firestore is schemaless, so the shape is enforced at the two ends instead: the `COLLECTIONS` constant is the single source of truth for every collection name (a typo is otherwise a silently empty read, which looks like "no data"), and TypeScript plus Zod guard the write boundary.

Conventions that are load-bearing rather than stylistic:

- **Root collections** for anything the console filters across — `drivers` , `deliveries` , `payroll_*` , `notification_*` , `esign_*` , `visit_*` .
- **Subcollections only for true children** — import batches → rows, service records → attachments.
- **Denormalised list fields** on the parent, so a list page needs no join: driver name, code and employee id are copied onto `deliveries` .
- `uniq_*` **lock documents** emulate partial unique indexes for MG ID, phone, plate and civil ID. A lock is released when the owner is archived, so an identity is never reserved by a record nobody can see.
- `counters/{name}` documents back the human-readable code sequences (`EMP-…` , `RCM-…` , `SIG-…` , `VIS-…` , `BAT-…`).
- **Kuwait-day values are** `YYYY-MM-DD` **strings**, so "today" is an equality filter rather than a timezone computation the index cannot use.
- **Rollups are written absolutely** (`rollups_driver_day` , `rollups_zone_month`) with SET, never increment, so a replay cannot double count.

4.3 Domain groups (142 collections)

Group	Examples
People and HR	`drivers` , `driver_intakes` , `profiles` , `driver_documents` , `attendance_logs` , `driver_daily_shifts` , `driver_off_structure`

Group	Examples
Deliveries and ops	`deliveries`, `delivery_verifications`, `order_recon_rows`, `order_recon_runs`, `rider_compliance_daily`, `fleet_events`, `driver_locations`, `driver_location_events`
Payroll	`payroll_clients`, `payroll_client_rules`, `payroll_day_hours`, `payroll_zone_metrics`, `payroll_order_overrides`, `payroll_rule_audit_logs`, `driver_payouts`
Earnings and rules	`driver_earnings_daily`, `delivery_rules`, `delivery_rule_scopes`, `incentive_rules`, `incentive_rule_tiers`, `incentive_rule_scopes`
Fleet	`vehicles`, `asset_catalog`, `asset_assignments`, `fuel_fills`, `driver_groups`
Employee Desk	`requests`, `request_*`, `visit_bookings`, `visit_slots`, `visit_branches`, `esign_*`
Notifications	`notification_campaigns`, `notification_dispatch_items`, `notification_device_tokens`, `notification_automations`
Platform	`app_settings`, `admin_permissions`, `admin_roles`, `page_read_snapshots`, `page_read_generations`, `admin_activity_logs`

4.4 Indexes and read economics

An index is not a formality here: Firestore bills per document read, so a query that degrades to a collection scan costs money per keystroke. Three patterns carry most of the saving:

- **Keyset pagination** on the fields the list actually sorts by, so a page costs a page of reads.
- **Snapshot reads** (`page_read_snapshots`) behind the expensive scans — payroll months, HR bundles, order comparison — with a generation counter that invalidates them on write.
- `count()` **aggregates** instead of fetching documents to count them.

`docs/FIRESTORE_COST_GUARDRAILS.md` holds the read model, the caps and the expiry policy; `docs/perf/` holds the measured baselines.

4.5 Legacy database

A PostgreSQL / Supabase project (`eoksxkdssptgyqyywdju`) was the original store. It is **paused**, and neither the console nor the Worker calls it. Its 398 migrations remain in `supabase/migrations` as history only. The cutover order and its verification criteria are recorded in `docs/CUTOVER.md`.

4.6 File storage

Store	Bucket	Content
Cloudflare R2	`dpd-private-prod`	Driver documents, eSign PDFs and signatures, request attachments, partner logos
Cloudflare R2	`dpd-gps-prod`	GPS archive written by the `dpd-live` Worker

Files are private by default: the browser receives a short-lived presigned URL, and a key is signed only when a record actually references it. A key that no record points at is not readable even if it is guessed.

5 Hosting and deployment

5.1 Hosting matrix

Layer	Platform	Identity	Region
Admin console	**Vercel**	project `dpdadmin-prod`	`bom1` (Mumbai)
Production URL	—	`https://dpdadmin-prod.vercel.app`	—
Application backend	**Cloud Functions gen 2**	`musallam-delivery-prod`	`me-central1`
Database	**Firestore**	`musallam-delivery-prod` / `default`	`me-central2`
Edge services	**Cloudflare Workers**	account `04c485fa…`	Global edge
Object storage	**Cloudflare R2**	`dpd-private-prod` , `dpd-gps-prod`	Middle East
Auth / push	**Firebase Auth**, **FCM**	`musallam-delivery-prod`	`me-central2`
Error tracking	**Sentry**	org `vizsoft-global` , project `dpd-admin`	SaaS

5.2 Regions and data residency

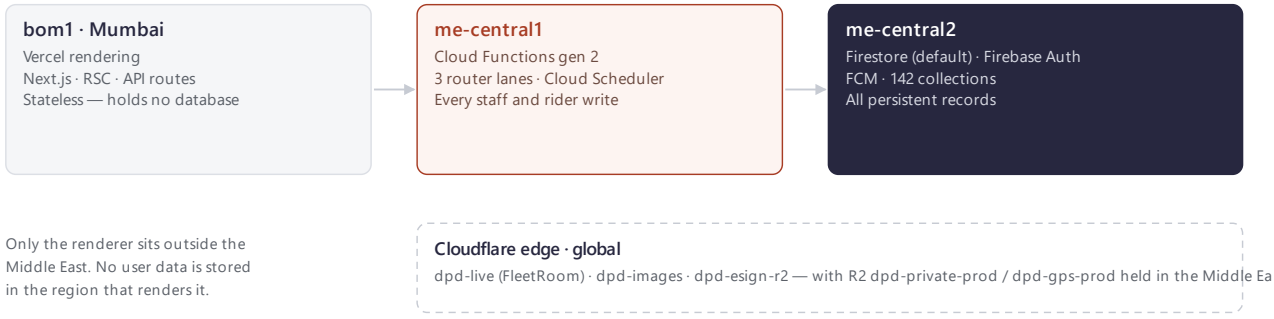


Figure 6. Regions and residency. Persistent data and the compute that touches it stay in the Middle East; Vercel is a stateless renderer in front of them.

All persistent data — Firestore, both R2 buckets, and the compute that touches them — sits in the Middle East. Vercel renders in Mumbai and holds no database of its own; it is a stateless renderer in front of `me-central1` . That is why a console page can be slow on a poor link without any data being at risk, and why the region choice is stated rather than implied.

5.3 Vercel configuration

```
{
  "framework": "nextjs",
  "regions": ["bom1"],
  "crons": [ 13 scheduled jobs ]
}
```

- Node runtime **24.x**; build command `next build`; output tracing includes the eSign font directory and the Chromium binary the PDF route needs.
- The environment is **production-only**: local `.env.local`, Vercel Production and Vercel Preview all point at the same live project. There is no staging clone of the database, so destructive testing is never performed against it.
- Deployments are built from a **clean worktree**, so an unrelated in-flight branch cannot ship with a change. The alias is then verified rather than assumed — `vercel inspect` must resolve `dpadmin-prod.vercel.app` to the new deployment with `target production` and `status Ready`, and the routes are smoked (`/en/login` 200; authenticated routes 307 at the auth gate, which is a redirect and not a 404).

5.4 Cloud Functions deployment

- `firebase deploy --only functions:<name>` — **one name at a time, never `--force`**, so a failure is attributable to a name.
- **A routed callable is not live until its lane is deployed.** The three lanes are separate services; deploying the callable alone changes a URL nobody calls.
- Scheduled functions run from Cloud Scheduler in `me-central1`.
- A functions deploy is followed by an unauthenticated probe: the callable must answer 401, not 200 and not HTML.

5.5 Release checklist

1. `tsc --noEmit` on the root project **and** on `functions`.
2. `npm run build` on a clean `.next`.
3. `npm run test:i18n` plus the suites the change touches.
4. Deploy functions (by lane where routed), then Vercel production.
5. `vercel inspect` the alias; smoke `/en/login` and the changed routes.
6. Commit and push; the change log entry names the deployment id.

6 Rider application (MG-GO)

A separate Flutter codebase, Android-only, distributed through Google Play, and talking to the same Firebase backend as the console.

6.1 Runtime and language

Item	Value
Package / product	`dpd_userapp` — "Musallam Delivery Partner"
Language	**Dart** — pubspec `sdk: ^3.11.0`, lockfile resolves `>=3.12.0 <4.0.0`
Framework	**Flutter 3.44.0 stable** (engine `4c525dac`, DevTools 2.57.0)
Platform	**Android only** — `applicationId com.musallam_delivery.app`, `arm64-v8a`
Android toolchain	AGP **8.11.1**, Kotlin **2.2.20**, Google Services **4.4.2**, Java **17**, core-library desugaring
`minSdk` / `targetSdk`	Flutter toolchain defaults — deliberately not pinned in the project
App version	**1.1.26+118** (`versionName` 1.1.26, `versionCode` 118)
Distribution	**Google Play only** — no flavors, no sideload or APK OTA; the app refuses to run with Android Developer options enabled
Code size	**298 Dart files** across `lib/core` (24 subsystems) and `lib/features` (21 features); **109 test files**

6.2 Architecture inside the app

- **go_router 17.2** for navigation, **flutter_riverpod 3.3** for state, in a feature-first layout where each `lib/features/*` folder owns its data, domain and UI.
- **Offline-first.** `sqflite` holds a local database at **schema version 10**; writes are queued and drained when `connectivity_plus` reports the network is back, so a delivery completed in a dead zone is not lost.
- **Localisation built in.** ARB files compiled by `flutter gen-l10n` — `app_en.arb` and `app_ar.arb`, **1,259 keys each**, full RTL.
- **Brand fonts bundled** (Gotham for Latin, GE SS TWO for Arabic) — no runtime font fetch, so the first frame is the final frame.

6.3 Packages in use

Concern	Packages
Firebase	`firebase_core` 4.2, `firebase_auth` 6.1, `cloud_firestore` 6.1, `cloud_functions` 6.0, `firebase_messaging` 16.0
Location and duty	`geolocator` 14.0, `flutter_foreground_task` 9.1, `permission_handler` 12.0, `disable_battery_optimization` 1.1, `battery_plus` 6.2, `flutter_device_compass` 2.0
Camera and on-device ML	`camera` 0.11, `image_picker` 1.2, `google_mlkit_face_detection` 0.13, `google_mlkit_text_recognition` 0.15

Concern	Packages
Local storage	`sqflite` 2.4, `shared_preferences` 2.5, `path_provider` 2.1
Documents	`pdfrx` 2.4 (in-app eSign viewing), `qr_flutter` 4.1, `flutter_svg` 2.2
Observability	`sentry_flutter` 9.20 + `sentry_dart_plugin` — project `flutter-mussalam`
UX	`flutter_local_notifications` 19.4, `app_links` 7.0, `package_info_plus` 9.0, `url_launcher` 6.3, `video_player` 2.9, `google_fonts` 8.1

6.4 Security and integrity controls

- **Passcode login, not email/password.** A rider signs in with the driver code plus a 6-digit passcode issued by the console (Verify & approve → regenerate passcode). Firebase Auth holds the resulting session.
- **Single-device session.** A heartbeat guards the session: a second login raises a conflict naming the active device, and a device that is replaced gets a short grace window to flush queued writes before it is signed out.
- **Developer-options refusal** at boot; there is no sideload path to fall back to.
- **Check-in selfie** gated on the assigned restaurant or zone, evaluated on-device with ML Kit face detection.
- **Order-photo OCR** on-device, then re-checked server-side against the console's *Order photo OCR check* setting.
- **Attachments** go through the admin API's presigned-upload route; the app never holds bucket credentials.

6.5 Backend integration

Item	Value
Firebase project	**`musallam-delivery-prod`** (project number `579224507592`)
Functions region	`FIREBASE_FUNCTIONS_REGION` dart-define, default **`me-central1`**
Admin API base	`ADMIN_API_BASE_URL` dart-define, default **`https://dpdadmin-prod.vercel.app`**
Configuration	`env/prod.json` via `--dart-define-from-file`; a non-production admin host is refused
Push	**FCM** plus local notifications
Live position	the `dpd-live` Worker (WebSocket) and the `driver*` callables

6.6 Build and release

- Scripts: `run_prod.sh`, `build_prod.sh`, `build_play.sh`, `release.sh`, `verify_apk_signing.sh`, `check_release_keystore.sh`.
- Release builds are signed with the production keystore (`android/key.properties`). Gradle throws rather than emit an unsigned release, and `applicationId` plus signing must never change, so Play and MDM upgrades stay valid.
- `sentry_dart_plugin` uploads debug symbols and source maps on release.
- References: `docs/RELEASE_PROCESS.md`, `docs/PASSCODE_LOGIN.md`, `docs/TWO_STAGE_DELIVERY.md`, `docs/admin/single-device-session-handoff.md`.

6.7 Tests

`flutter test` over **109 test files**, guarded by `flutter analyze`. The Arabic RTL overflow suites load the real GE SS TWO and Gotham faces rather than a fallback, so a width regression in Arabic fails a test instead of surprising a rider.

7 Engineering standards

7.1 Types and guards

- `strict` TypeScript on both projects; `tsc --noEmit` is part of every release.
- Discriminated unions are switched exhaustively with a `never` check, so a new variant fails to compile until it is handled.
- Validation happens twice on purpose: Zod on the client for a useful message, and an explicit server guard for the truth.

7.2 Tests

`node --test` suites grouped per domain (`test:payroll`, `test:drivers`, `test:vehicles`, `test:service`, `test:esign`, `test:i18n`, …), plus `flutter test` for the app. Suites pin behaviour rather than implementation: a label map, a rate precedence rule, an i18n key, a queue rule.

7.3 Change control

Every structural change appends a dated entry to `AGENTS.md` and, where the rider app is affected, to `docs/DRIVER_APP_HANDOFF.md` in the same commit. The entry names what moved, why, the verification that was run, and the deployment id — so a future reader can tell a claim from a measurement.

8 Source code and documentation

8.1 Source delivery

Each application has its own shared folder. Open the folder for the code you need; each holds the full working tree **and** a downloadable archive of it.

Application	Source	Archive
Admin console (web)	MGgo-Admin	`ADMIN_source_code.zip` — 15.2 MB
Rider app (Android)	MGgo-User	`USER_source_code.zip` — 5.8 MB

Application	Technology
Admin console (web)	Next.js 16 · React 19 · TypeScript
Rider app (Android)	Flutter 3.44 · Dart 3.12

Both applications are delivered in the same release, so the console and the app in these two folders are the pair that belongs together. Access is by link sharing on the folders, and the archives are byte-identical copies of the tree beside them — use either.

8.2 Delivered manuals

Four documents, delivered as PDF:

Document	Scope	Pages
Admin Panel Manual	Complete administrator manual — every console module, module by module	34
User Manual	Rider app end to end, plus the console companion view	39
Employee Bulk Import	Client-facing import procedure and column reference	—
Technical Stack Overview	This document	25

The same four files are also built into the repository under `docs/`, together with their markdown sources. `npm run docs:admin`, `docs:manual` and `docs:stack` regenerate them, so a correction reaches the shared folder on the next build rather than being retyped into a copy.

8.3 Architecture documents

Document	Contents
`docs/CUTOVER.md`	Production architecture, migration order, rollback per step, external writer contracts
`docs/DRIVER_APP_HANDOFF.md`	The complete admin ↔ rider API contract — every callable, payload and edge case
`docs/FIRESTORE_COST_GUARDRAILS.md`	Read-cost model, caps, snapshot strategy, expiry policy

Document	Contents
`docs/FLEET_SERVICE_MAINTENANCE.md`	Per-programme maintenance model, baseline import, odometer sync
`docs/DESIGN_SYSTEM.md`	Tokens, typography, spacing and colour semantics
`docs/DRIVER_OPS_COVERAGE.md`	Which driver actions are audited, and which are deliberately not
`docs/CHANGELOG.md`	Dated engineering change log
`firestore.rules`	Authoritative security rules
`firestore.indexes.json`	Authoritative index schema
`src/lib/firebase/db.ts`	Authoritative collection-name schema
`functions/router-inventory.json`	Machine-readable inventory of every function, lane, memory, CPU and timeout

The rider app carries its own notes in `MG-GO/docs/`: `RELEASE_PROCESS.md`, `PASSCODE_LOGIN.md`, `TWO_STAGE_DELIVERY.md`, `FLAVOR_INTEGRATION.md`, `LOGIN_VERIFICATION_HANDOFF.md`, and under `docs/admin/` `single-device-session-handoff.md` plus `server-verification-checklist.md`.

8.4 API surface

There is no hand-written OpenAPI file, because a generated one would be a third copy that drifts. The contract is expressed in three authoritative places which together are complete:

1. `functions/src/rpcs/*.ts` — **106 modules**, each a typed callable with payload validation.
2. `functions/src/router/lanes.ts` and `router-inventory.json` — which lane serves each callable, with its runtime limits.
3. `docs/DRIVER_APP_HANDOFF.md` — the rider-facing subset, written as a contract for the Flutter team.

9 Provenance

9.1 Where each figure was read

Claim	File
Framework and library versions	`package.json`, `node_modules/*/package.json`
Node engine, Vercel node version	`package.json`, `.nvmrc`, `.vercel/project.json`
Console areas (41)	`src/app/[locale]/(dashboard)/`
API route groups (15)	`src/app/api/`
Cron jobs (13)	`vercel.json`
Functions runtime and project	`firebase.json`, `.firebaserc`, `functions/package.json`
Function count (204), lanes (167 / 11 / 26)	`functions/router-inventory.json`, `functions/src/router/lanes.ts`
RPC modules (106)	`functions/src/rpcs/*.ts` (test files excluded)
Firestore database, edition, region	`docs/CUTOVER.md`, `firebase.json`
Collection schema (142)	`src/lib/firebase/db.ts`
Indexes (201) and overrides (5)	`firestore.indexes.json`
Permission slugs (172 code / 174 synced)	`src/lib/auth/permission-catalog.ts`, `admin_permissions/catalog`
i18n keys (12,591 per locale)	`src/messages/en.json`, `src/messages/ar.json`
Sentry org and project	`next.config.ts`
Worker names, bindings, compat dates	`infra/workers/*/wrangler.jsonc`
Vercel region, project, crons	`vercel.json`, `.vercel/project.json`
Legacy Supabase state	`docs/CUTOVER.md`, `supabase/migrations/`
Flutter and Dart SDK versions	`MG-GO/pubspec.yaml`, `MG-GO/pubspec.lock`
Rider app version, `applicationId`, ABI	`MG-GO/pubspec.yaml`, `MG-GO/android/app/build.gradle.kts`
Android toolchain (AGP / Kotlin / Google Services)	`MG-GO/android/settings.gradle.kts`
Dart files (298), test files (109)	`MG-GO/lib/`, `MG-GO/test/`
ARB keys (1,259 per locale)	`MG-GO/lib/l10n/app_en.arb`, `app_ar.arb`
Offline schema (v10)	`MG-GO/lib/core/offline/*.dart`
Single-device session behaviour	`MG-GO/lib/features/auth/device_session_models.dart`
Developer-mode hard block	`MG-GO/lib/core/security/*.dart`

9.2 Document control

Field	Value
Version	**1.5**
Date	11 November 2026
Prepared from	the `MG-GO-Admin` and `MG-GO` working copies
Reader	client technical stakeholders, onboarding engineers, procurement
Scope	stack, architecture, hosting and provenance — **not** a user manual
Rebuild	`npm run docs:stack` (markdown → PDF, figures and plates included)
Diagrams	`docs/assets/architecture/*.jpg` — the two module dependency maps

What changed from v1.3:

- **§8.1 links each application's own source folder** — MGgo-Admin and MGgo-User — with the archive name and size beside each, instead of one folder covering three products.
- **§8.2 links every delivered PDF**, so the manual set can be opened from this document rather than hunted for in a Drive folder.

What changed from v1.2:

- **The two detailed module dependency maps are replaced by two high-level architecture diagrams** — one for the admin console, one for the rider app. The dependency maps carried several hundred nodes and were unreadable at A4; they are removed from the document, and the two new figures are drawn from the source at the level of layers and managed services.
- **The source-code link is the current shared folder** — *Source Codes MG*, holding the **MG Hub**, **Bajaj Care** and **Bites** products. §8.1 now names which folder is this platform.
- The manual page counts in §8.2 were re-read from the built PDFs (`ADMIN_PANEL_MANUAL` **34**, `USER_MANUAL` **39**); this document states its own **25**.

What changed from v1.1:

- Added **the two module dependency maps** (admin console and rider app) as landscape plates, generated from the working copies — they show which file depends on which, which the layer diagram cannot.
- Every count was re-derived from the tree. Corrections: collections **148** → **142**, composite indexes **206** → **201** (plus 5 field overrides), deployed functions **207** → **204** with the lane split published (167 / 11 / 26), RPC modules **158** → **106** (the earlier figure counted test files), console areas **42** → **41**, i18n keys **~12,600** → **12,591**, permission slugs published as **172 in code / 174 synced**.
- Added: the layered architecture diagram, the request-path diagram with the lane router, the live-tracking path, and the region map.
- Added: the full 13-job cron table, the four account kinds with what each resolves to, the index/read-economics section, the engineering-standards section, and the release checklist.
- Clarified: a routed callable is not live until its lane is deployed.